4. 딥러닝 시작

goblurry · 약 2시간 전

통계 수정 삭제

딥러닝

딥러닝 파이토치 교과서

목록 보기

1/1

📊 딥러닝 파이토치 교과서 4장을 바탕으로 학습한 내용을 정리한 포스트입니다.

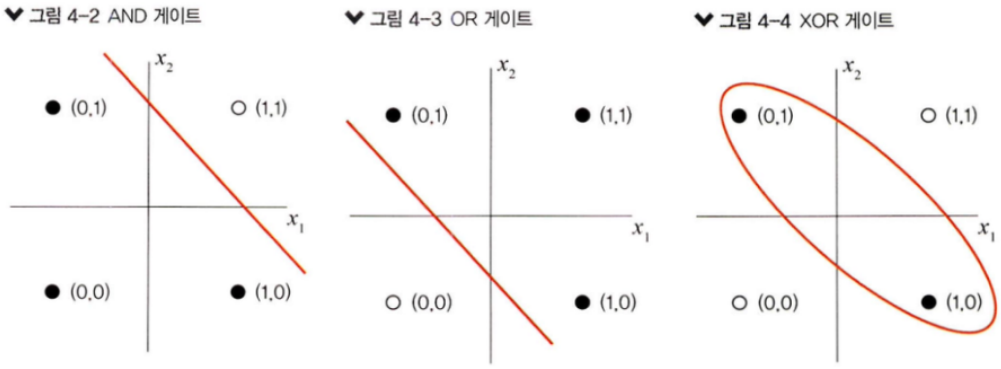
> 1. 인공 신경망의 한계와 딥러닝의 출현
> 2. 딥러닝 구조
> 3. 딥러닝 알고리즘
> 4. 우리는 무엇을 배워야 할까?

4.1 인공 신경망의 한계와 딥러닝의 출현

이 절에서는 인공 신경망의 한계를 극복하기 위해 고안된 **딥러닝**의 개념을 소개한다.

인공 신경망에서 이용하는 구조는 **퍼셉트론**이라는 **선형 분류기**이다. 퍼셉트론은 다수의 입력을 받아 하나의 신호를 출력한다.

입력이 2개 있을 때, 컴퓨터가 이를 논리적으로 인식하는 방식에는 크게 AND, OR, XOR(논리적 배타합)이 있다.



AND 게이트는 모든 입력이 1일 때 1을 출력하고, 위 이미지와 같이 데이터를 분류한다. 검은 점과 흰 점을 분류하는 것이 선형임을 확인할 수 있다.

OR 게이트는 둘 중 하나라도 1일 때 1을 출력한다. 마찬가지로 선형적으로 분류된다.

문제가 되는 것은 XOR 게이트(논리적 배타합)이다. 두 입력 중 하나만 1일 때 1을 출력하게 되는데, 위 그림과 같이 검은 점(출력=0)과 흰 점(출력=1)이 비선형적으로 분류된다.
이 경우 데이터를 제대로 분류하기가 어렵다.

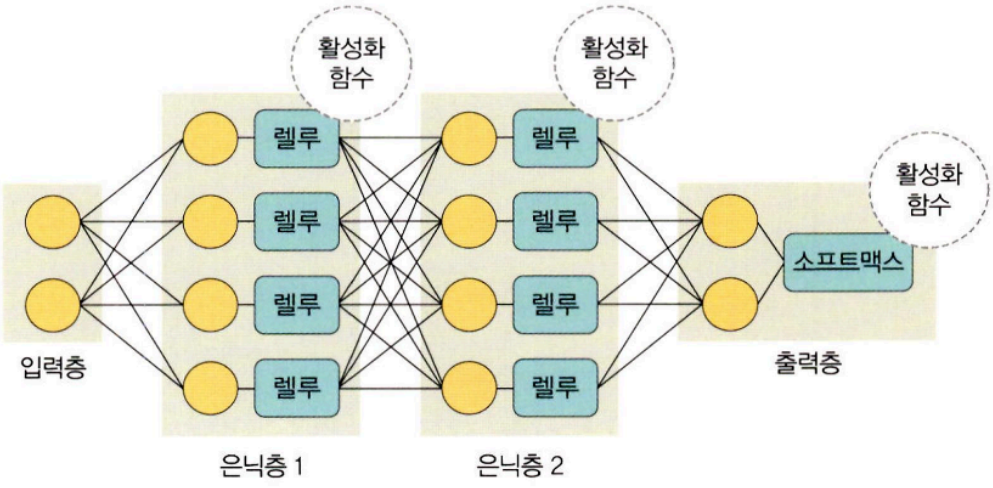
퍼셉트론에서는 AND, OR 연산에 대해서는 학습이 가능하나 XOR 연산에 대해서는 학습이 불가능해서, 이를 극복하기 위한 방법으로 **다중 퍼셉트론**(Multi-layer Perceptron)을 고안했다.

이는 입력층과 출력층 사이에 보이지 않는 하나 이상의 중간층(=은닉층)을 두어, 비선형적으로 분리되는 데이터에 대해서도 학습이 가능하도록 한 것이다.

이처럼 입력층과 출력층 사이에 여러 개의 은닉층이 존재하는 신경망을 **심층 신경망(Deep Neural Network, DNN)** 이라고 하며, 이를 다른 이름으로 딥러닝이라고 하는 것이다.

4.2 딥러닝 구조

> 🌟 딥러닝: 여러 층을 가진 인공신경망을 사용해 학습을 수행하는 것



딥러닝의 구성 요소

- **층**
  - 입력층: 데이터를 입력받는 층
  - 은닉층: 모든 입력 노드부터 입력값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용해 출력층에 전달하는 층
  - 출력층: 신경망의 최종 결괏값이 포함된 층
- **가중치(w)**: 노드 간의 연결 강도
- **바이어스(b)**: 가중합에 더해지는 상수로, 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 '조절하는' 역할
- **가중합**(weighted sum), 전달 함수: 가중치*신호
- 함수
  - **활성화 함수**: 신호를 입력받아 이를 적절히 처리해 출력하는 함수
  - **손실 함수**: 가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수

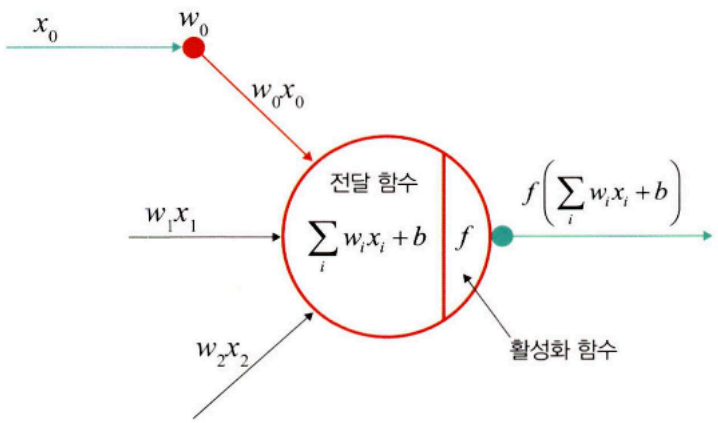
① 가중치

가중치는 입력값의 연산 결과를 조정하는 역할을 한다. 입력값에 가중치를 곱해 가중합을 구하고, 이를 활성화 함수에 입력해 출력값을 얻기 때문에 가중치의 값이 얼마인지에 따라 출력값이 달라진다.

② 가중합 (=전달 함수)

각각의 입력 노드에 대해, 입력값*가중치 값들을 전부 더한 것이다. 이 가중합을 활성화 함수로 보내기 때문에 전달 함수라고도 한다.

그림 4-7 전달 함수



가중합을 구하는 공식은 다음과 같습니다.

$$\sum_i w_i x_i + b$$

(w: 가중치, b: 바이어스)

실제로 활성화 함수의 입력값으로 사용되는 값은 입력값 * 가중치 한 값에 바이어스 값을 더한 것이다.

(교재에서 위와 같이 가중합을 구하는 공식에 바이어스 값을 포함해 두어서, 가중합 자체는 입력값에 가중치를 곱한 값이고 바이어스 값은 별개인 것이 아닌가 했는데 찾아보니 실무에서는 바이어스 값도 가중치처럼 취급하고, 가중합에 이를 포함하는 경우도 많다고 한다.)

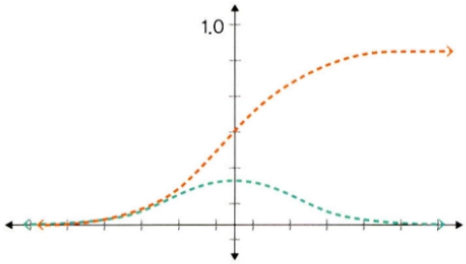
③ 활성화 함수

활성화 함수는 전달 함수에서 계산된 값
(∑wixi+b) 을 입력으로 받아, 이를 변환하여 뉴런의 최종 출력값을 만드는 비선형 함수이다.

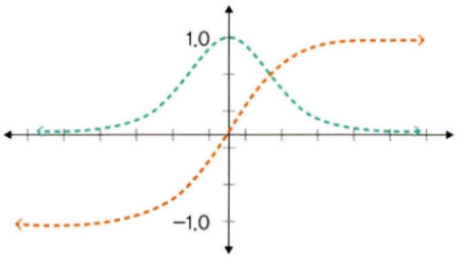
즉, 입력값에 가중치를 곱해 얻은 가중합에 바이어스를 더한 값을 활성화 함수에 넣어 최종 출력값을 계산한다.

대표적인 활성화 함수로는 시그모이드, 하이퍼볼릭 탄젠트(tanh), ReLU, 소프트맥스 함수 등이 있다.

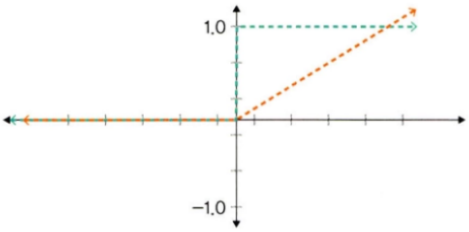
❤ 그림 4-8 시그모이드 활성화 함수와 미분 결과



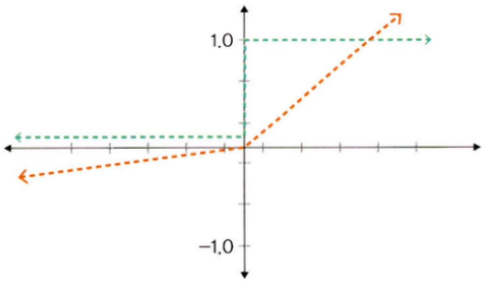
❤ 그림 4-9 하이퍼볼릭 탄젠트 활성화 함수와 미분 결과



❤ 그림 4-10 렐루 활성화 함수와 미분 결과



❤ 그림 4-11 리키 렐루 활성화 함수와 미분 결과



(주황 선은 활성화 함수, 녹색 선은 미분 결과 도함수)

1. 시그모이드 함수

시그모이드 함수는 선형 함수를 0-1 사이에서 비선형 형태로 변환한다. 분류 문제(ex. 로지스틱 회귀)를 확률적으로 표현할 때 사용한다.

그러나, 딥러닝 모델의 깊이가 깊어지면 '기울기 소멸 문제(vanishing gradient)'가 발생하여 딥러닝 모델에서 잘 사용하지 않는다.

*기울기 소멸 문제는 역전파 과정에서 기울기 값이 계속 작아져 앞쪽 층으로 갈수록 값이 거의 0이 되어 가중치가 제대로 업데이트되지 않는 현상이다.

2. 하이퍼볼릭 탄젠트 함수

선형 함수를 -1에서 1 사이에서 비선형 형태로 변환한다. 시그모이드 함수 활용 시 발생하던 결괏값의 평균이 양수로 편향되는 문제를 해결했으나 기울기 소멸 문제는 여전히 발생한다.

3. ReLU 함수

렐루 함수는 최근 활발히 사용되는 활성화 함수로, 주로 은닉층에서 사용된다. 입력값이 음수일 때는 0, 양수일 때는 입력값을 출력한다.

기울기 소멸 문제가 발생하지 않고, 학습 속도가 빠르다는 장점이 있다.(경사 하강법에 영향을 주지 않음)

4. Leaky ReLU 함수

렐루 함수의 경우, 입력값이 음수일 때 항상 0을 출력하기 때문에 학습 능력이 감소한다는 문제가 있다. 이를 해결하기 위해 고안된 함수가 Leaky ReLU(리키 렐루) 함수이다. 리키 렐루 함수에서는 입력 값이 음수이면 0이 아닌 0.001과 같은 아주 작은 수를 반환한다. 그 결과, 입력값이 수렴하는 구간이 제거되어 렐루 함수에서 발생하던 문제가 해결된다.

5. 소프트맥스 함수

입력값을 0-1 사이에 출력되도록 정규화하여 출력값들의 총합이 항상 1이 되도록 한다. 보통 출력 노드의 활성화 함수로 많이 사용된다. 수식은 다음과 같다.

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^{n} \exp(a_i)}$$

exp(a)는 지수함수를 의미하며, n은 출력층의 뉴런 개수이다. yk는 그중 k번째 출력값이다. 수식에 따르면, yk는 k번째 뉴런의 출력값 ak(가중합 값)의 지수값을 모든 출력 뉴런의 지수값 합으로 나눈 값으로 계산된다.

한마디로 각 값에 지수를 씌우고, 이를 다 더한 값으로 나누는 것이다. 이 과정을 통해 모든 출력값의 합은 1이 되며, 각 출력값을 확률로 해석할 수 있다.

④ 손실 함수

> **경사 하강법**: 학습률(1회 학습 시 변화를 얼마큼 주는지에 대한 상수)과 손실 함수의 순간 기울기를 활용해 가중치를 업데이트하는 방법. 미분의 기울기를 이용해 오차를 비교하고 최소화함.

경사 하강법 적용 시에 손실 함수를 이용해 오차를 구한다.
학습을 통해 얻은 데이터의 예측 값이 실제 데이터 값과 얼마나 차이나는지 평가하는 지표로, 값이 작을수록 데이터 추정의 정확도가 높다.

대표적인 손실 함수로 MSE(평균 제곱 오차, Mean Squared Error)와 CEE(Cross Entropy Error)가 있다.

1. 평균 제곱 오차, Mean Squared Error
(실제 값-예측 값), 즉 error를 제곱해 평균 낸 값을 평균 제곱 오차라고 한다. 값이 작을수록 모델의 예측력이 좋다. 주로 회귀에서 손실 함수로 사용된다.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$\left(\begin{array}{l} \hat{y}_i\text{: 신경망의 출력(신경망이 추정한 값)} \\ y_i\text{: 정답 레이블} \\ i\text{: 데이터의 차원 개수} \end{array}\right)$$

```
# 2. 평균 제곱 오차로 손실함수 정의
loss_fn = torch.nn.MSELoss(reduction='sum') # 오차 제곱값의 합 더하는 과정만 수행. 평균 x (기본값은 'mean')

# 예측 데이터 수행
y_pred = model(x)

# 정답 y와 예측 값을 비교해 손실함수에 대입, 손실값 계산
loss = loss_fn(y_pred, y)
```

2. 크로스 엔트로피 오차, Cross Entropy Error
*분류 문제에서 원핫 인코딩을 처리한 경우에만 사용 가능한 오차 계산법
분류 문제에서 시그모이드 함수 사용 시 수식에 포함된 자연상수 e로 인해 MSE를 적용하면 매끄럽지 않은 그래프가 출력된다. 크로스 엔트로피 손실 함수를 사용하면 경사 하강법 과정에서 학습이 지역 최소점에서 중단되어 해당 문제를 방지할 수 있다.

0

딥러닝 학습

딥러닝 학습은 순전파와 역전파 두 단계로 이루어진다.

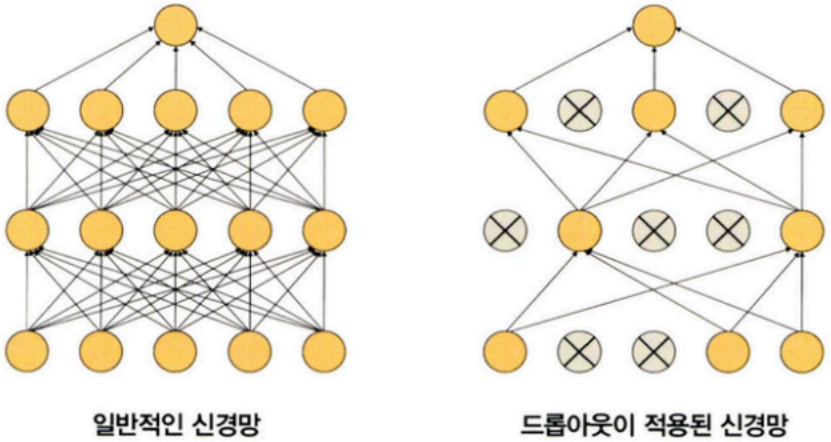
- **순전파** (Feedforward): 네트워크에 입력된 훈련 데이터를 기반으로 예측 값을 계산하기 위해 전체 신경망을 거쳐 가는 단계. 이전 층의 뉴런에서 수신한 정보에 변환(가중합과 활성화 함수)을 적용해 다음

4.1 인공 신경망
현
4.2 딥러닝 구
딥러닝의 구
딥러닝 학습
딥러닝의 문
4.3 딥러닝 알
심층 신경망
Network
합성곱 신경

층의 뉴런으로 전송하는 방식.

- **역전파** (Backpropagation): 손실 함수로 오차를 추정할 때, 오차를 최대한 0에 가깝도록 하기 위해 모델이 훈련을 반복하며 가중치를 조절한다. 계산된 손실(오차)을 바탕으로 출력층→은닉층→입력층 순으로 역으로 전파하는 방식이다. (각 뉴런에서 예측 값과 실제 값의 차이를 가중치로 미분한 후 기존 가중치 값에서 빼는 과정을 모든 뉴런에 대해 진행한다. 그 결과 계산된 뉴런의 결과를 다시 순전파의 가중치로 사용한다.)

그림 4-15 일반적인 신경망과 드롭아웃이 적용된 신경망



딥러닝의 문제점과 해결 방안

활성화 함수를 적용한 여러 개의 은닉층을 결합해 비선형 영역을 표현하는 것이 딥러닝의 핵심이었다. 은닉층의 개수가 늘어날수록 데이터가 잘 분류된다.

하지만 무조건 좋은 것만은 아닌데, (1) 과적합 (2) 기울기 소멸 (3) 성능 악화 - 이 세 문제의 발생 가능성이 있다.

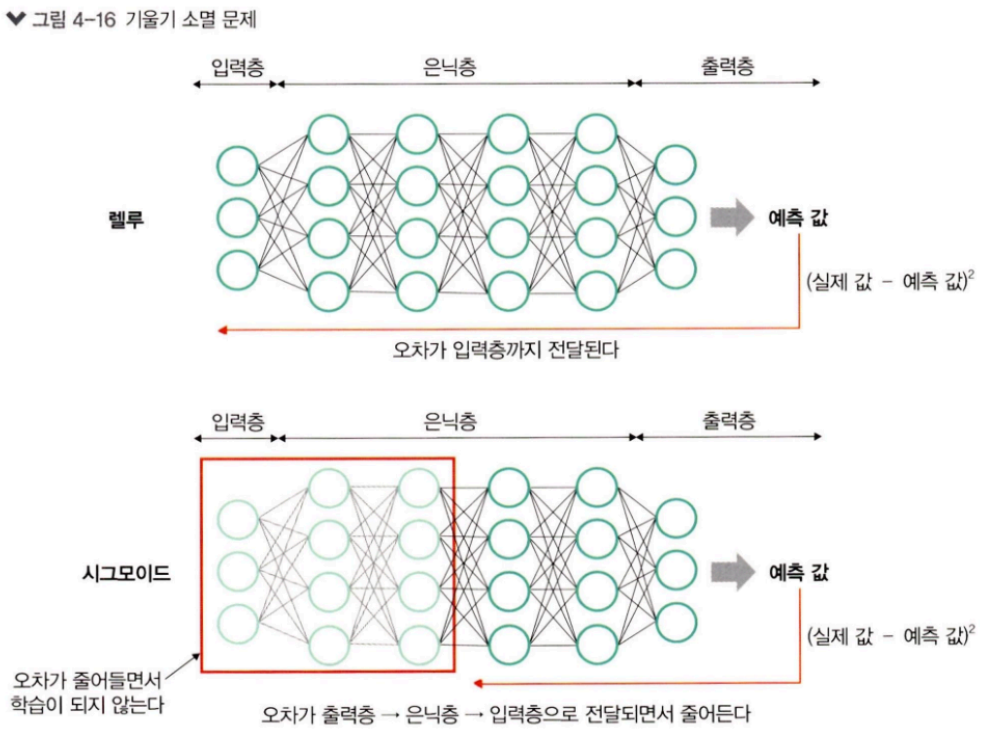
① 과적합 문제 → 드롭아웃

훈련 데이터를 과하게 학습해서 예측 값과 실제 값 사이 오차는 줄이지만 '실제 데이터'에 대해서 오차가 증가할 수 있는 문제이다. 이를 해결하는 방안으로 **드롭아웃**이 있다.

드롭아웃: 학습 과정 중 임의로 일부 노드를 학습에서 제외시키는 방식 !

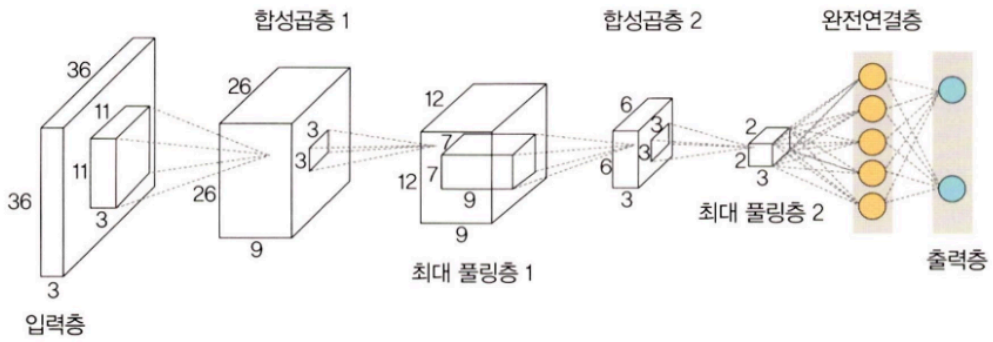
② 기울기 소멸 문제 → ReLU 함수

출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 학습이 되지 않는 현상으로, 은닉층의 개수가 많은 신경망에서 주로 발생한다.
기울기가 소멸되기 때문에 학습량이 0에 가까워져 오차를 줄이지 못한 상태로 수렴하는 것이다. 앞서 언급했던 활성화 함수 중 시그모이드와 하이퍼볼릭 탄젠트에서 주로 발생하며, 이 대신 ReLU 함수를 사용하면 해결 가능하다.



③ 성능 악화 문제 → 확률적 경사 하강법 및 미니 배치 경사 하강법

오차가 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 경사 하강법을 반복할 경우 성능이 나빠진다.



이 문제를 개선한 방법들로 다음 두 가지가 있다.

1. 확률적 경사 하강법 (Stochastic Gradient Descent)
임의로 선택한 데이터에 대해 기울기를 계산하는 방법. 데이터가 적어 계산 속도가 빨라진다. 단, 배치 경사 하강법보다 정확도가 낮을 수 있고, 파라미터 변경 폭이 불안정하다.

2. 미니 배치 경사 하강법 (Mini-batch Gradient Descent)
전체 데이터셋을 미니 배치 여러 개로 나누고, 배치 1개당 기울기를 구한 후 평균을 구해 모델을 업데이트해 학습하는 방법이다.
전체 데이터의 기울기를 계산하는 배치 경사 하강법보다는 계산 속도가 빠르며, 확률적 경사 하강법보다는 파라미터 변경 폭이 안정적이기 때문에 실제로 가장 많이 사용된다.

딥러닝의 이점

- **특성 추출**: 데이터별로 가지는 특징을 찾아내고, 그 특징을 토대로 데이터 → 벡터 변환을 수행하는 과정. ML 알고리즘(주로 SVM, Naive Bayes, LR)에서의 특징 추출은 매우 복잡하고 수집 데이터에 대한 전문 지식이 필요했던 반면, 딥러닝에서는 특성 추출 과정이 알고리즘에 통합되었다.
- **빅데이터의 효율적 활용**: 특성 추출이 알고리즘에 통합될 수 있었던 이유가 빅데이터 때문이다. 데이터 사례가 많을수록 딥러닝 모델의 성능이 향상되기 때문이다.

4.3 딥러닝 알고리즘

모든 딥러닝 알고리즘의 공통점은 **심층 신경망**을 사용한다는 것이며,

- 합성곱 신경망 (CNN)
- 순환 신경망 (RNN)
- 제한된 볼츠만 머신 (RBM)
- 심층 신뢰 신경망 (DBM)

목적에 따라 위의 네 가지로 분류된다.

심층 신경망 Deep Neural Network

> 🌟 입력층과 출력층 사이에 여러 개의 은닉층이 존재하는 신경망 (4.1 참고)

- 장점: 다양한 비선형적 관계를 학습할 수 있음.
- 단점: 연산량 많음. 기울기 소멸 문제 발생 가능.
  → 해결 방법: 드롭아웃, ReLU 함수, 배치 정규화 등

합성곱 신경망 CNN

Convolutional Neural Network의 약자로, 합성곱층과 풀링층(Pooling layer)을 포함하는 고성능 **이미지 처리 인공 신경망 알고리즘**이다.

영상과 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체의 위치를 찾아내는 데 유용하다.

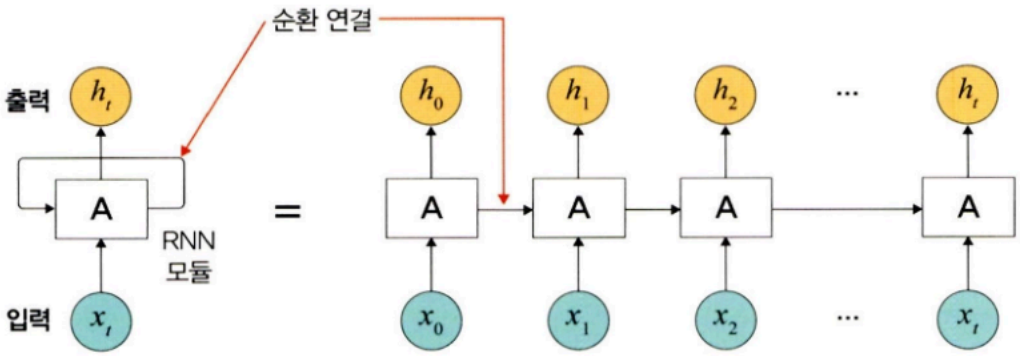
- 대표적인 CNN: LeNet-5, AlexNet
- 층을 더 깊게 쌓은 신경망: VGG, GoogLeNet, ResNet

(이후 5장과 6장에서 자세히 다룬다고 한다.)

순환 신경망 RNN

Recurrent Neural Network의 약자로, **시계열 데이터**를 학습하기 위한 인공 신경망이다. 자기 자신을 참조하며, 현재 결과가 이전 결과와 연관이 있다.

❮ 그림 4-26 순환 신경망



RNN의 특징

- 시간성을 가진 데이터가 많음.
- 시간성 정보를 이용해 데이터의 특징을 다룸.
- 데이터가 동적이고 길이가 가변적임.
- 기울기 소멸 문제로 제대로 학습되지 않는 현상 발생 → 메모리 개념을 도입한 LSTM (long short term memory)가 사용되고 있음.
- 자연어 처리에 효과적임: 언어 모델링, 텍스트 생성, 기계 번역, 음성 인식 등

(순환 신경망은 교재의 7장에서 다룬다고 한다.)

제한된 볼츠만 머신 RBM

Restricted Boltzmann Machine. 볼츠만 머신은 가시층과 은닉층으로 구성된 모델인데, 이 모델에서 가시층과 은닉층만이 연결되는 모델을 제한된 볼츠만 머신이라고 한다. 즉, 가시층-가시층, 은닉층-은닉층 연결이 없는 것이다.

RBM의 특징

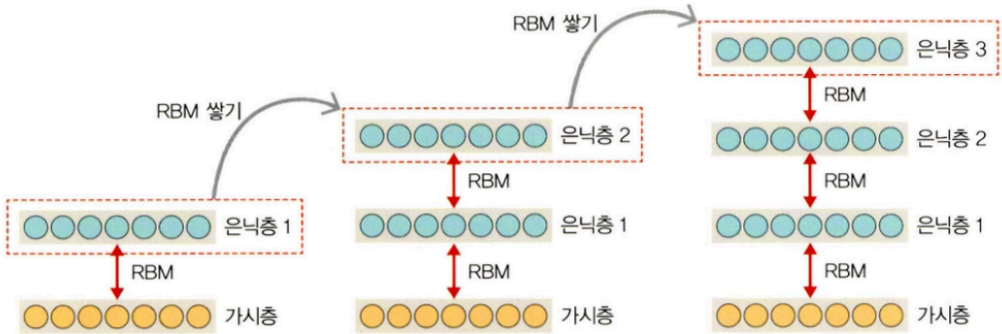
- 기울기 소멸 문제 해결을 위한 사전 학습 용도로 활용 가능함.
- 심층 신뢰 신경망의 요소로 활용됨.
- 차원 감소, 분류, 선형 회귀 분석, 협업 필터링, 특성 값 학습, 주제 모델링 등에 사용함.

*딥러닝에서 주로 쓰이는 알고리즘은 CNN과 RNN이다.

심층 신뢰 신경망 DBN

Deep Belief Network의 약자로, 위의 제한된 볼츠만 머신을 여러 층으로 쌓아 연결된 신경망이다.
사전 훈련을 완료한 RBM을 쌓아 올려 레이블이 없는 데이터에 대한 비지도 학습이 가능하다.

❮ 그림 4-28 심층 신뢰 신경망



DBN의 학습 절차

1. 가시층과 은닉층 1에 RBM을 사전 훈련 한다.
2. 첫 번째 층 입력 데이터와 파라미터를 고정해 두 번째 층 RBM을 사전 훈련 한다.
3. 원하는 층의 개수만큼 RBM을 쌓아 올려 DBN을 완성한다.

DBN의 특징

- 순차적으로 DBN을 학습시켜 가며 계층 구조를 생성함.
- 비지도 학습으로 학습함.
- 위로 올라갈수록 추상적 특성을 추출함.
- 학습된 가중치를 다층 퍼셉트론의 가중치 초깃값으로 사용함.

정리하면, 복잡한 비선형 데이터에 대한 분류 및 예측을 수행하기 위해서는 딥러닝 학습이 필요하다.

goblurry

0개의 댓글

댓글을 작성하세요

댓글 작성

관심 있을 만한 포스트

Powered by
Stellate